

Lilian Weng 这篇经典文章 [《LLM Powered Autonomous Agents》](#) 不是一篇严格意义上的学术论文，而是一篇高质量技术综述/博客，但因为结构清晰、覆盖面广、提出了后续 Agent 系统的核心框架，所以在 Agent 领域非常经典。

这篇文章的核心观点是：**大语言模型可以作为 Agent 的“大脑”，但真正可用的自主 Agent 不能只靠模型本身，而要由 Planning、Memory、Tool Use 三类能力共同支撑。**

1. Agent 系统的整体框架

文章开头给出了一个 LLM-powered autonomous agent 的总体架构。核心组件包括：

组件	作用	通俗解释
LLM	核心控制器 / 大脑	负责理解任务、推理、决策
Planning	规划能力	把复杂任务拆成步骤，并安排执行顺序
Memory	记忆能力	保存历史信息、经验、上下文
Tool Use	工具使用	调用搜索、代码执行、数据库、API 等外部能力
Action	行动	根据计划和工具结果执行下一步
Reflection	反思	复盘错误，调整后续策略

文章第 2 页的系统图非常关键：它把 Agent 看成一个中枢系统，LLM 位于中心，周围连接 Memory、Planning、Tools 和 Action。这个图基本可以看作现代 Agent 框架的原型图。

如果用金融业务类比：

LLM 像一个战略分析师，Planning 是它做项目计划的能力，Memory 是它查历史材料和过往决策的能力，Tool Use 是它调用 Wind、彭博、内部数据库、Python、搜索引擎的能力，Reflection 是它复盘为什么上一次判断错了。

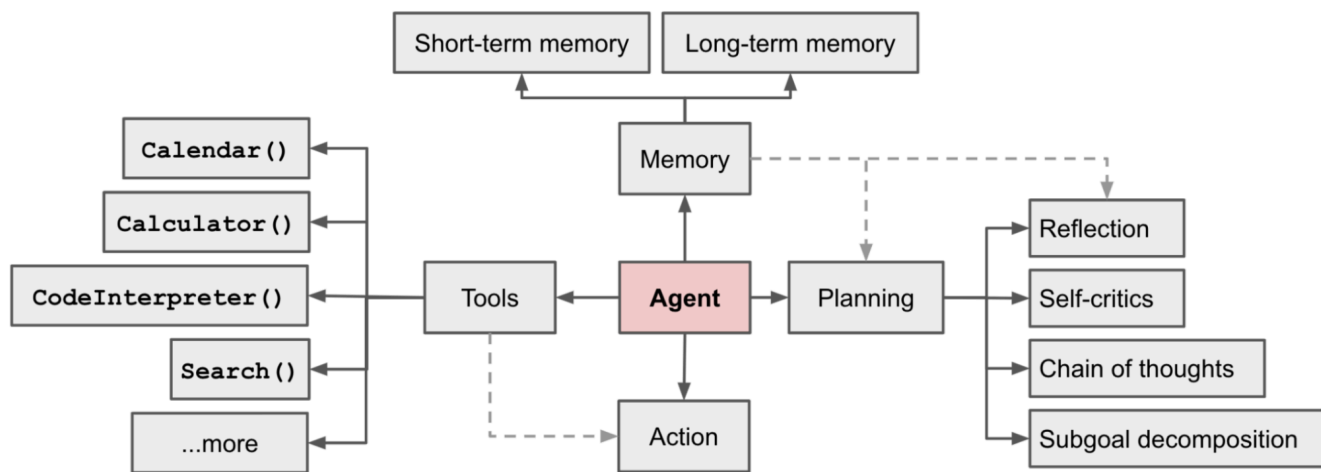


Figure 1: Overview of a LLM-powered autonomous agent system.

2. 第一大组件：Planning 规划能力

Planning 是 Agent 区别于普通 Chatbot 的核心能力之一。

普通 Chatbot 往往是：

用户问一句 → 模型答一句。

Agent 则是：

用户给一个目标 → Agent 自己拆任务 → 逐步执行 → 根据反馈调整 → 最终完成目标。

文章把 Planning 分成两个方向：

2.1 Task Decomposition: 任务拆解

复杂任务往往不能一步完成，需要拆成若干子任务。

例如用户说：

帮我做一个“研报问答 RAG 系统”。

普通模型可能直接输出一堆建议。

Agent 应该先拆解：

1. 明确目标用户和使用场景；
2. 收集研报 PDF；
3. 做文档解析；
4. chunk 切分；
5. embedding；

6. 建立向量库；
7. 检索；
8. rerank；
9. 引用溯源；
10. 问答生成；
11. 权限与风控；
12. 评估与监控。

这就是任务拆解。

文章提到几个代表方法：

Chain of Thought, CoT

CoT 的核心是让模型“step by step”思考。它把复杂问题拆成中间步骤，从而提升推理效果。

通俗理解：

不要让模型直接报答案，而是让它先列出解题过程。

这对复杂数学题、逻辑题、规划题都有帮助。

Tree of Thoughts, ToT

Tree of Thoughts 是 CoT 的扩展。CoT 是一条思路，ToT 是多条思路。

它会在每一步生成多个可能的 reasoning path，然后用搜索方法，例如 BFS 或 DFS，选择更优路径。

通俗理解：

CoT 像一个人沿着一条路往前走；ToT 像一个人同时探索几条路，比较哪条更可能成功。

这对开放式问题很有价值，比如写作、策略规划、复杂决策。

LLM+P

LLM+P 的思路是：LLM 不一定自己完成所有规划，可以把规划问题转成 PDDL（PDDL = Planning Domain Definition Language，把现实里的**任务、动作、规则、目标**写成机器能看懂的规范文本），然后交给传统规划器求解。

这说明一个重要思想：

Agent 不一定要让 LLM 干所有事情。**LLM 更适合做自然语言理解、任务转换、结果解释**；严谨规划或计算可以交给专门工具。

这和现代 Agent 工程非常一致：LLM 负责决策和编排，工具负责确定性执行。

2.2 Self-Reflection：自我反思

文章强调，真实任务中 trial and error 是不可避免的，所以 Agent 需要反思能力。

这部分提到几个重要框架。

ReAct：Reasoning + Acting

ReAct 是 Agent 领域非常重要的范式。它把推理和行动结合起来，典型格式是：

```
Thought: 我需要先查找相关信息
Action: Search(...)
Observation: 搜索结果返回.....
Thought: 根据结果,我还需要进一步验证.....
Action: Browse(...)
Observation: 页面内容.....
```

它的关键不在于“模型说了 Thought”，而在于形成了一个循环：

思考 → 行动 → 观察 → 再思考 → 再行动。

这也是你现在做 Market Insight Agent 时非常重要的基础：

不是让 LLM 一次性写结论，而是让它在数值事实、异常检测、工具搜索之间循环。

Reflexion

Reflexion 进一步引入“失败后的反思记忆”。

它的逻辑是：

1. Agent 执行任务；
2. 如果失败，生成一段 reflection；
3. 把 reflection 存入 memory；
4. 下一次执行同类任务时参考这段经验。

例如一个 Agent 做网页购物任务失败了，它可能反思：

我之前没有确认商品是否加入购物车，下次在结账前必须检查购物车状态。

这就从单次行动升级为跨轮次学习。

Chain of Hindsight, CoH

CoH 的核心思想是：让模型看到一系列带反馈的历史输出，从较差答案到较好答案，然后学习如何基于反馈改进。

通俗理解：

给模型看“差答案 → 人类反馈 → 更好答案”的过程，让它学会自我修正的趋势。

这和写作、代码生成、报告生成中的迭代优化非常接近。

3. 第二大组件：Memory 记忆能力

文章把 Agent 的 Memory 类比为人类记忆系统。

3.1 人类记忆和 Agent 记忆的对应关系

文章中提到三类人类记忆：

人类记忆	Agent 中的对应	解释
Sensory Memory 感觉记忆	原始输入的 embedding 表征	对文本、图像、语音等输入做向量化
Short-term Memory 短期记忆	Context Window	当前对话上下文，容量有限
Long-term Memory 长期记忆	外部向量库 / 数据库	可长期保存和检索信息

这部分非常重要，因为它解释了为什么 Agent 不能只依赖上下文窗口。

LLM 的上下文窗口再长，也有边界。真实 Agent 系统需要处理大量历史任务、文档、用户偏好、环境状态，因此必须有外部记忆。

通俗说：

Context window 是模型眼前摊开的几页纸；Vector Store 是它的资料库；检索系统是它从资料库里找相关内容的能力。

3.2 Long-term Memory 和向量检索

文章提到，长期记忆通常通过 embedding + vector store 实现。

基本流程是：

- 1. 把文本、事件、历史经验转成 embedding；
- 2. 存入向量数据库；
- 3. 当 Agent 需要相关信息时，把当前 query 也转成 embedding；
- 4. 通过相似度搜索找回最相关的历史信息。

这就是 RAG 和 Agent 记忆系统的共同基础。

文章还介绍了 MIPS，即 Maximum Inner Product Search，用于快速找相似向量。常见近似最近邻算法包括：

算法	通俗解释
LSH	用哈希把相似内容分到相同桶里
ANNOY	用随机投影树快速缩小搜索范围
HNSW	用小世界图构建多层索引，快速跳转到近邻
FAISS	Facebook 开源的高性能向量检索库
ScaNN	Google 的高效向量搜索方案

这些算法本质上解决的是：

向量库里可能有几百万、几千万条记忆，Agent 不可能每次全量扫描，必须快速找到最相关的几条。

对你关心的金融场景来说，这一点非常关键。比如一个研报问答 Agent，如果背后有几十万篇研报，就必须依赖高效索引、召回和 rerank，否则系统会慢且不准。

4. 第三大组件：Tool Use 工具使用

Tool Use 是这篇文章的另一个核心。

文章认为，工具使用是人类智能的重要特征。LLM 本身虽然强大，但它有明显局限：

- 1. 模型参数里的知识可能过时；
- 2. 不能天然访问私有数据；
- 3. 不擅长精确计算；
- 4. 不能直接执行代码；

- 5. 不能直接操作外部系统；
- 6. 不能可靠感知实时环境。

所以 Agent 必须能调用外部工具，比如：

工具	作用
Search API	获取最新信息
Calculator	做精确计算
Code Interpreter	执行代码
Database	查询结构化数据
Calendar API	操作日程
Browser	浏览网页
Internal System API	调用企业内部系统
Vector Store	检索长期记忆

通俗理解：

LLM 是大脑，工具是手脚和感官。没有工具的 LLM 只能“想”；有工具的 Agent 才能“做”。

4.1 MRKL：LLM 作为路由器

MRKL 是 Modular Reasoning, Knowledge and Language 的缩写。

它的思想是：系统由多个专家模块组成，LLM 负责判断应该调用哪个模块。

例如：

用户问题	应调用模块
“今天美元兑人民币是多少？”	汇率 API
“帮我算一下 IRR”	计算器 / Python
“查一下某公司最新公告”	搜索 / 数据库
“总结这篇 PDF”	文档解析 + LLM
“生成图表”	Python / 可视化工具

这体现了 Agent 系统中的一个关键原则：

不要让 LLM 做它不擅长的确定性任务，而是让它学会把任务路由给合适的工具。

4.2 Toolformer / TALM：让模型学会调用工具

文章提到 Toolformer 和 TALM，它们关注的是如何通过训练让模型学会在合适的时候插入 API 调用。

这和简单 prompt 调工具不同。它们希望模型内部学到：

1. 什么时候需要工具；
2. 需要哪个工具；
3. 如何构造参数；
4. 如何利用工具返回结果继续生成答案。

这其实是现代 Function Calling / Tool Calling 的前身思想。

4.3 HuggingGPT：LLM 调度多个模型

HuggingGPT 是一个典型的多工具 / 多模型 Agent 案例。

它用 ChatGPT 作为中央调度器，调用 Hugging Face 上的不同模型完成不同任务。

文章总结了它的四个阶段：

1. **Task Planning**：把用户请求拆成多个任务；
2. **Model Selection**：为每个任务选择合适模型；
3. **Task Execution**：调用模型执行；
4. **Response Generation**：汇总执行结果，生成最终答案。

通俗理解：

HuggingGPT 像一个项目经理，自己不一定亲自画图、识别语音、生成视频，但它知道该把哪个子任务分配给哪个专家模型。

这对理解“Agent 不是单模型，而是调度系统”非常重要。

4.4 API-Bank：评估工具使用能力

API-Bank 是一个评估 Tool-Augmented LLM 的 benchmark。

它关注 Agent 是否能完成完整 API 工作流，包括：

- 1. 判断是否需要 API；
- 2. 找到正确 API；
- 3. 阅读 API 文档；
- 4. 构造正确参数；
- 5. 调用 API；
- 6. 根据结果决定是否继续调用；
- 7. 最终回答用户。

文章把工具使用能力分成三个层级：

层级	能力	通俗解释
Level 1	给定 API，能正确调用	会用一个已知工具
Level 2	能检索合适 API 并学习使用	会找工具、读说明书
Level 3	能规划多个 API 调用完成复杂任务	会组织一串工具链完成目标

这对企业 Agent 特别重要。真正有价值的 Agent 往往不是调用一个工具，而是能编排多个工具。

比如“安排一次出差”可能涉及：

- 1. 查日程；
- 2. 查航班；
- 3. 查酒店；
- 4. 查预算；
- 5. 发邮件；
- 6. 创建日历；
- 7. 报销系统预填。

这就是 Level 3。

5. 案例研究

文章后半部分列举了几个代表性 Agent 案例。

5.1 Scientific Discovery Agent

文章提到 ChemCrow 和 Boiko 等科学发现 Agent。

这类 Agent 可以调用化学数据库、文献搜索、代码执行、实验设备 API 等工具，辅助完成药物发现、材料设计等任务。

关键启发是：

在专业领域，Agent 的能力不只来自 LLM，而是来自 LLM + 专业工具 + 专业数据库 + 领域约束。

这对金融同样适用。

金融 Agent 不能只靠通用模型，需要连接：

1. 行情数据；
2. 研报库；
3. 公告库；
4. 财务数据库；
5. 内部客户数据；
6. 风险规则；
7. 合规边界；
8. 估值模型；
9. 回测工具。

否则它只能给出泛泛而谈的回答。

文章也特别提醒了风险：在科学领域，Agent 可能被用于危险化学品、生物武器等高风险任务。这说明 Agent 能力越强，安全边界越重要。

5.2 Generative Agents

Generative Agents 是一个非常经典的模拟实验：25 个由 LLM 控制的虚拟角色生活在沙盒环境中，类似 The Sims。

每个 Agent 都有：

1. Memory stream：记录经历；
2. Retrieval model：按相关性、时间新近性、重要性检索记忆；
3. Reflection mechanism：从具体记忆中总结高层认知；
4. Planning and reacting：根据记忆、计划和环境采取行动。

这个案例最有意思的是出现了“涌现式社会行为”：

1. 信息扩散；

2. 关系记忆；
3. 对话延续；
4. 多人协作；
5. 社交活动组织。

通俗理解：

当每个虚拟人都有记忆、反思和计划时，他们的行为就不再像随机 NPC，而更像有连续人格和社会关系的人。

这说明 Agent 的“智能感”很大程度来自记忆和状态连续性，而不只是单轮回答质量。

5.3 AutoGPT

AutoGPT 是早期非常有影响力的 Agent Demo。

它的核心形式是：

1. 用户设定目标；
2. Agent 自己生成计划；
3. 调用搜索、文件、代码等工具；
4. 保存长期记忆；
5. 不断执行直到目标完成。

但文章也指出，AutoGPT 有很多可靠性问题，尤其是自然语言接口导致的格式解析问题。

这点非常现实：

很多早期 Agent 的代码，真正复杂的地方不是“智能”，而是如何让模型稳定输出可解析 JSON、如何处理失败、如何重试、如何防止跑偏。==

这和你现在做 Agent 工程时遇到的 schema 校验、JSON 输出、工具参数、降级机制，本质上是同一个问题。

5.4 GPT-Engineer

GPT-Engineer 是另一个重要案例：用户用自然语言描述要做的软件，Agent 先提澄清问题，再生成完整代码仓库。

它的流程体现了几个工程原则：

1. 不直接开写；
2. 先澄清需求；
3. 拆分模块；
4. 明确文件结构；
5. 输出完整代码；
6. 尽量保证可运行。

这和 **Agentic Coding** 的基本模式非常接近。

通俗理解：

GPT-Engineer 的目标不是回答“怎么写代码”，而是像一个初级工程师一样，帮你把需求变成一个可运行项目。

6. 文章最后总结的核心挑战

文章最后提出了 LLM Agent 的几个关键限制。直到今天，这些限制依然成立。

6.1 有限上下文长度

LLM 的 context window 是有限的。Agent 需要历史任务、工具返回、文档内容、错误日志、用户偏好、系统约束等大量上下文，但模型无法一次性全部看完。

向量数据库可以缓解这个问题，但不能完全替代 full attention。

通俗说：

RAG 能帮模型“查资料”，但查出来的几段资料不等于模型真正完整理解了所有历史。

这也是为什么长期任务 Agent 很难。

6.2 长期规划困难

LLM 在短任务上表现很好，但长周期、多步骤、开放环境任务仍然困难。

主要问题包括：

1. 任务拆解不稳定；
2. 遇到错误时不善于重规划；
3. 容易陷入无效循环；
4. 难以判断当前路径是否偏离目标；
5. 缺乏可靠的全局状态管理。

这也是为什么真实 Agent 系统需要：

1. workflow 约束；
 2. 状态机；
 3. 任务队列；
 4. tool schema；
 5. 中间结果校验；
 6. 人类审批节点。
-

6.3 自然语言接口不可靠

文章指出，当前 Agent 系统大量依赖自然语言作为 LLM 与工具、记忆、环境之间的接口。但自然语言输出天然不稳定，模型可能：

1. 格式错误；
2. JSON 不合法；
3. 参数缺失；
4. 编造工具名称；
5. 忽略系统约束；
6. 拒绝执行本该执行的指令；
7. 输出不可解析内容。

所以现代 Agent 工程必须引入：

1. function calling；
2. JSON Schema；
3. structured output；
4. 参数校验；
5. retry；
6. fallback；
7. observability；
8. sandbox；
9. permission control。

也就是说：

Agent 工程的关键不是写一个炫酷 prompt，而是把 LLM 的不确定输出放进一个可控、可验证、可恢复的系统里。

7. 这篇文章对 Agent 的核心贡献

我认为这篇文章的价值不在于提出某一个新算法，而在于它系统性地总结了 LLM Agent 的基本范式。

它把 Agent 拆成了三个核心能力：

7.1 Planning

回答的是：

Agent 如何把大目标拆成可执行步骤？

包括 CoT、ToT、LLM+P、ReAct、Reflexion 等。

7.2 Memory

回答的是：

Agent 如何记住历史、经验和外部知识？

包括短期上下文、长期向量库、MIPS、ANN 检索等。

7.3 Tool Use

回答的是：

Agent 如何突破模型自身能力边界，真正和外部世界交互？

包括 MRKL、Toolformer、HuggingGPT、API-Bank 等。

这三个模块合起来，构成了早期 LLM Agent 的主流架构。

8. 用一个通俗例子理解：Agent 像一个会做项目的实习生

假设你让 Agent 完成任务：

帮我做一份关于券商机构业务收入结构的分析报告。

一个普通 LLM 会直接生成报告。

一个 Agent 应该这样做：

Step 1: Planning

它先拆任务：

1. 明确报告框架；
2. 收集数据；
3. 查找券商年报；
4. 提取机构业务收入；
5. 对比现金交易、PB、衍生品、做市；
6. 生成图表；
7. 写结论；
8. 校验引用来源。

Step 2: Memory

它会调用历史记忆：

1. 你以前关注 Goldman Sachs / Morgan Stanley equities revenue；
2. 你关心现金、衍生、PB、做市拆分；
3. 你偏好表格化、结构化、可用于内部汇报的风格；
4. 你要求不要夸张，要中性客观。

Step 3: Tool Use

它会调用：

1. 搜索工具查最新年报；
2. PDF 解析工具抽取收入表；
3. Python 做数据清洗和图表；
4. Excel/Word 工具生成文件；
5. 引用系统做溯源。

Step 4: Reflection

它发现某家券商没有披露细分收入，于是反思：

1. 不能硬估；
2. 应标注“未披露”；
3. 可用业务描述和行业口径做推断，但要明确假设；
4. 最终报告中区分“事实”和“推断”。

这才是一个真正的 Agent，而不是一次性文本生成器。

9. 对你学习 Agent 的启发

结合你现在从金融战略转向 AI Agent / Solution Architect，这篇文章有几个特别重要的学习抓手。

9.1 不要把 Agent 理解成“Prompt 写得更复杂”

Agent 的本质不是 prompt，而是系统架构。

Prompt 只是其中一层。更重要的是：

1. 状态如何维护；
2. 工具如何注册；
3. 参数如何校验；
4. 记忆如何检索；
5. 任务如何拆解；
6. 失败如何恢复；
7. 权限如何控制；
8. 输出如何评估。

9.2 Agent 的核心是“闭环”

一个 Agent 至少要有：

```
Goal → Plan → Act → Observe → Reflect → Re-plan → Final Output
```

如果系统只是：

```
Input → LLM → Output
```

那它更像 Chatbot，不是 Agent。

9.3 工具调用是工程落地的关键

企业 Agent 的价值不在于“会说”，而在于“能接系统”。

比如金融场景下，Agent 需要接：

1. 研报库；
2. 行情库；
3. 交易数据；
4. 客户数据；

5. 权限系统；
6. 审批系统；
7. Excel/Word/PPT 输出；
8. 邮件和日程；
9. 风控规则。

这要求你不仅懂 LLM，也要懂系统集成、数据契约、权限控制和流程设计。

9.4 Memory 是长期任务的关键

没有 Memory，Agent 每次都像第一次见你。

有 Memory，Agent 才能形成连续性：

1. 记得你的偏好；
2. 记得过去任务；
3. 记得错误经验；
4. 记得业务背景；
5. 记得项目状态。

但 Memory 不能盲目塞上下文，必须有检索、过滤、排序和失效机制。

9.5 可靠性比“聪明”更重要

文章最后对自然语言接口不可靠的讨论非常关键。

在 demo 里，Agent 看起来很惊艳；在生产环境里，真正决定可用性的往往是：

1. JSON 是否稳定；
2. 工具参数是否正确；
3. API 错误是否能重试；
4. 是否会陷入循环；
5. 是否能安全降级；
6. 是否能解释证据来源；
7. 是否能被监控和审计。

这正是 Agent 从 demo 走向 production 的核心难点。

10. 这篇文章的知识地图

可以把全文浓缩成下面这张结构：

LLM Powered Autonomous Agent

- |
 - └─ 1. Planning
 - |└─ Task Decomposition
 - |└─ CoT
 - |└─ Tree of Thoughts
 - |└─ LLM+P
 - |└─ Self-Reflection
 - |└─ ReAct
 - |└─ Reflexion
 - |└─ Chain of Hindsight
 - |└─ Algorithm Distillation
 - |
 - └─ 2. Memory
 - |└─ Short-term Memory = Context Window
 - |└─ Long-term Memory = Vector Store
 - |└─ MIPS / ANN
 - |└─ LSH
 - |└─ ANNOY
 - |└─ HNSW
 - |└─ FAISS
 - |└─ ScaNN
 - |
 - └─ 3. Tool Use
 - |└─ MRKL
 - |└─ TALM
 - |└─ Toolformer
 - |└─ Function Calling
 - |└─ HuggingGPT
 - |└─ API-Bank
 - |
 - └─ 4. Case Studies
 - |└─ ChemCrow
 - |└─ Scientific Discovery Agent
 - |└─ Generative Agents
 - |└─ AutoGPT
 - |└─ GPT-Engineer
 - |
 - └─ 5. Challenges
 - |└─ Finite Context Length
 - |└─ Long-term Planning Difficulty
 - |└─ Natural Language Interface Reliability
-

11. 这篇文章为什么经典

这篇文章经典，是因为它在 2023 年比较早地把 LLM Agent 的关键问题讲清楚了：

1. **Agent 不是单纯的 LLM。**

LLM 只是核心控制器，必须配合规划、记忆和工具。

2. **Agent 的能力来自系统组合。**

模型能力、工具能力、记忆能力、反馈机制共同决定效果。

3. **Agent 的难点在可靠性。**

Demo 很容易，生产可用很难。

4. **Agent 是从“生成答案”走向“完成任务”。**

这是 Chatbot 到 Agent 的本质跃迁。

5. **Agent 工程天然是跨学科的。**

它同时涉及 LLM、检索、规划、工具调用、软件工程、系统设计、安全和评估。

最后总结

这篇文章可以看作 LLM Agent 的“总纲”。

它告诉我们：

一个真正的 Agent，不是更会聊天的模型，而是一个以 LLM 为核心控制器，具备规划、记忆、工具调用和反思能力的任务执行系统。

LLM 本身负责理解和推理；

Planning 负责把目标拆成路径；

Memory 负责提供历史和上下文；

Tool Use 负责连接外部世界；

Reflection 负责从失败中改进；

工程系统负责让这一切稳定、可控、可评估。

这就是 Agent 从概念走向真实生产系统的核心逻辑。